

Capítulo 20

Software de entrada/saída

20.1 Introdução

O sistema operacional é responsável por oferecer acesso aos dispositivos de entrada/saída às aplicações e, em consequência, aos usuários do sistema. Prover acesso eficiente, rápido e confiável a um conjunto de periféricos com características diversas de comportamento, velocidade de transferência, volume de dados produzidos/consumidos e diferentes interfaces de hardware é um enorme desafio. Além disso, como cada dispositivo define sua própria interface e modo de operação, o núcleo do sistema operacional deve implementar o código necessário para interagir com milhares de tipos de dispositivos distintos. Como exemplo, cerca de 60% das 20 milhões de linhas de código do núcleo Linux na versão 4.3 pertencem a código de *drivers* de dispositivos de entrada/saída.

Este capítulo apresenta uma visão geral da organização e do funcionamento dos componentes do sistema operacional responsáveis por interagir com os dispositivos de hardware de entrada/saída, e como o acesso a esses dispositivos é provido às aplicações em modo usuário.

20.2 Arquitetura de software de entrada/saída

Para simplificar o uso e a gerência dos dispositivos de entrada/saída, o código do sistema operacional é estruturado em camadas, que levam da interação direta com o hardware (como o acesso às portas de entrada/saída, interrupções e operações de DMA) às interfaces de acesso abstratas e genéricas oferecidas às aplicações, como arquivos e *sockets* de rede.

Uma visão conceitual dessa estrutura em camadas pode ser vista na Figura 20.1. Nessa figura, a camada inferior corresponde aos dispositivos físicos propriamente ditos, como discos rígidos, teclados, etc. A camada logo acima corresponde aos controladores de dispositivos (discos SATA, USB, etc.) e aos controladores de DMA e de interrupções implementados no *chipset* do computador.

A primeira camada de software no núcleo do sistema operacional corresponde aos *drivers* de dispositivos, ou simplesmente *drivers*¹, que são os componentes de código que interagem diretamente com cada controlador, para realizar as operações

¹Em Portugal se utiliza o termo “piloto”, que eu aprecio, mas optei por não utilizá-lo neste texto porque o termo em inglês é onnipresente no Brasil.

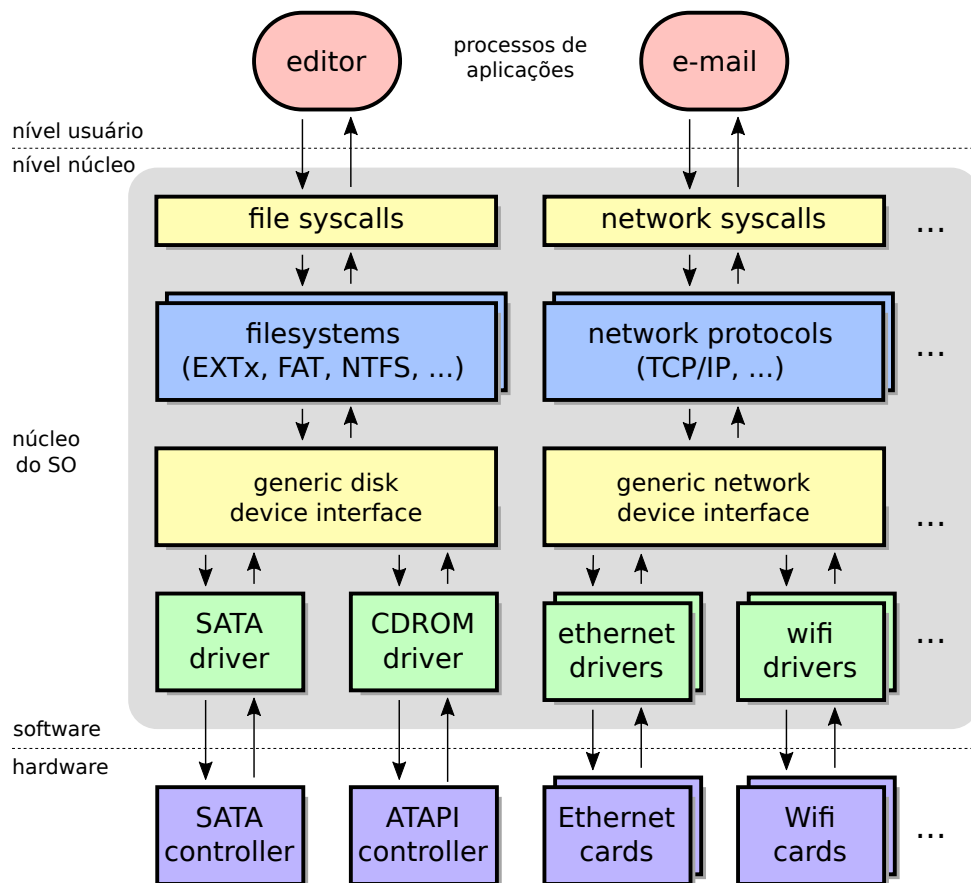


Figura 20.1: Estrutura em camadas do software de entrada/saída.

de entrada/saída, receber as requisições de interrupção e fazer o gerenciamento do dispositivo correspondente. Dentro de um mesmo grupo de dispositivos similares, como as placas de rede, há centenas de modelos de diferentes fabricantes, com interfaces distintas. Para cada dispositivo é então necessário construir um *driver* específico.

Acima dos *drivers* existe uma camada de código, denominada *generic device interface*, cuja finalidade é construir uma visão genérica de dispositivos similares, para que o restante do sistema operacional não precise ter consciência das peculiaridades de cada dispositivo, mas possa tratá-los por famílias ou classes, como dispositivos de armazenamento, interfaces de rede, de vídeo, etc.

Acima da camada de interface genérica de dispositivos, uma ou mais camadas de código estão presentes, para implementar abstrações mais complexas, como sistemas de arquivos e protocolos de rede. Finalmente, no topo da arquitetura de software, são implementadas as chamadas de sistema fornecidas às aplicações para acessar as abstrações construídas pelas camadas inferiores, como arquivos, diretórios e *sockets* de rede, etc.

20.3 Classes de dispositivos

Para simplificar a construção de aplicações e das camadas mais elevadas do próprio sistema operacional, os dispositivos de entrada/saída são geralmente agrupados em classes ou famílias com características similares, para os quais uma interface genérica

pode ser definida. Por exemplo, discos rígidos SATA, discos SSD e DVD-ROMs têm características mecânicas e elétricas distintas, mas servem basicamente para o mesmo propósito: armazenar arquivos. O mesmo pode ser afirmado sobre interfaces de rede *Ethernet* e *Wifi*: embora usem tecnologias distintas, ambas permitem a comunicação entre computadores.

Nos sistemas de padrão UNIX os dispositivos são geralmente agrupados em quatro grandes famílias²:

Dispositivos orientados a caracteres: são aqueles cujas transferências de dados são sempre feitas byte por byte, em sequência. Um dispositivo orientado a caracteres pode ser visto como um fluxo contínuo de entrada ou de saída de bytes. A característica sequencial faz com que não seja possível alterar o valor de um byte que já foi enviado. Dispositivos ligados às interfaces paralelas e seriais do computador, como *mouse* e teclado, são os exemplos mais clássicos desta família. Os terminais de texto e *modems* de transmissão de dados por linhas seriais (como as linhas telefônicas) também são considerados dispositivos orientados a caracteres.

Dispositivos orientados a blocos: são aqueles dispositivos em que as operações de entrada ou saída de dados são feitas usando blocos de bytes de tamanho fixo. Esses blocos são lidos ou escritos em posições específicas do dispositivo, ou seja, são endereçáveis. Conceitualmente, um dispositivo orientado a blocos pode ser visto como um vetor de blocos de bytes de mesmo tamanho. Discos rígidos, CDROMs, fitas magnéticas e outros dispositivos de armazenamento são exemplos típicos desta família.

Dispositivos de rede: estes dispositivos permitem enviar e receber mensagens entre processos e computadores distintos. As mensagens são blocos de dados de tamanho variável, com envio e recepção feitas de forma sequencial (não é possível alterar o conteúdo de uma mensagem que já foi enviada). As interfaces *Ethernet*, *Wifi*, *Bluetooth* e GPRS são bons exemplos desta classe de dispositivos.

Dispositivos gráficos: permitem a renderização de texto e gráficos em terminais de vídeo. Devido aos requisitos de desempenho, sobretudo para jogos e filmes, estes dispositivos exigem um alto desempenho na transferência de dados. Por isso, sua interface genérica é constituída por funções para consultar e configurar o dispositivo gráfico e uma área de memória compartilhada entre o processador e o dispositivo, usualmente denominada *frame buffer*, que permite acesso direto à memória de vídeo. Programas ou bibliotecas que interagem diretamente com o dispositivo gráfico têm acesso a essa área de memória através de bibliotecas específicas, como *DirectX* em ambientes Windows ou *DRI – Direct Rendering Engine* no Linux.

Vários dispositivos não se enquadram diretamente nas categorias acima, como receptores de GPS, sensores de temperatura e interfaces de áudio. Nestes casos, alguns sistemas operacionais optam por criar classes adicionais para esses dispositivos (como o Windows), enquanto outros buscam enquadrá-los em uma das famílias já existentes (como os UNIX). No Linux, por exemplo, os dispositivos de áudio são acessados pelas

²Nos sistemas *Windows* os dispositivos são agrupados em um número maior de categorias.

aplicações como dispositivos orientados a caracteres: uma sequência de bytes enviada ao dispositivo é tratada como um fluxo de áudio a ser reproduzido, geralmente em formato PCM. Cabe ao driver do dispositivo e à camada de interface genérica transformar essa interface orientada a caracteres nas operações de baixo nível necessárias para reproduzir o fluxo de áudio desejado.

20.4 Drivers de dispositivos

Um *driver* de dispositivo, ou simplesmente *driver*, é um componente do sistema operacional responsável por interagir com um controlador de dispositivo. Cada tipo de dispositivo possui seu próprio *driver*, muitas vezes fornecido pelo fabricante do mesmo. Cada *driver* é geralmente capaz de tratar um único tipo de dispositivo, ou uma família de dispositivos correlatos do mesmo fabricante. Por exemplo, o *driver* RTL8110SC(L), da empresa *Realtek Corp.*, serve somente para as interfaces de rede RTL8110S, RTL8110SB(L), RTL8169SB(L), RTL8169S(L) e RTL8169 desse fabricante.

Internamente, um *driver* consiste de um conjunto de funções que são ativadas pelo núcleo do sistema operacional conforme necessário. Existem basicamente três grupos de funções implementadas por um *driver*, ilustradas na Figura 20.2:

Funções de entrada/saída: responsáveis pela transferência de dados entre o dispositivo e o sistema operacional; essas funções recebem e enviam dados de acordo com a classe dos dispositivo: caracteres (bytes), blocos de tamanho fixo (discos), blocos de tamanho variável (pacotes de rede) ou áreas de memória compartilhadas entre o dispositivo e a CPU (imagens/vídeo e outros).

Funções de gerência: responsáveis pela gestão do dispositivo e do próprio *driver*. Além de funções para coordenar a inicialização e finalização do *driver* e do dispositivo, geralmente são fornecidas funções para configurar o dispositivo, para desligar ou colocar em espera o dispositivo quando este não for usado, e para tratar erros no *dispositivo*. Algumas dessas funções podem ser disponibilizadas aos processos no espaço de usuário, através de chamadas de sistema específicas como `ioctl` (UNIX) e `DeviceIoControl` (Windows).

Funções de tratamento de eventos: estas funções são ativadas quando uma requisição de interrupção é gerada pelo dispositivo. Conforme apresentado na Seção 19.6, toda requisição de interrupção gerada pelo dispositivo é encaminhada ao controlador de interrupções do hardware, que a entrega ao núcleo do sistema operacional. No núcleo, um tratador de interrupções (*IRQ handler*) reconhece e identifica a interrupção junto ao controlador e em seguida envia uma notificação de evento a uma função do *driver*, para o devido tratamento.

Além das funções acima descritas, um *driver* mantém estruturas de dados locais, para armazenar informações sobre o dispositivo e as operações em andamento.

Os *drivers* normalmente executam dentro do núcleo do sistema operacional, em modo privilegiado. Por ser código de terceiros executando com acesso total ao hardware, eles constituem um dos maiores riscos à estabilidade e segurança do sistema operacional. *Drivers* mal construídos ou mal configurados são fontes frequentes de problemas como travamentos ou reinicializações inesperadas.

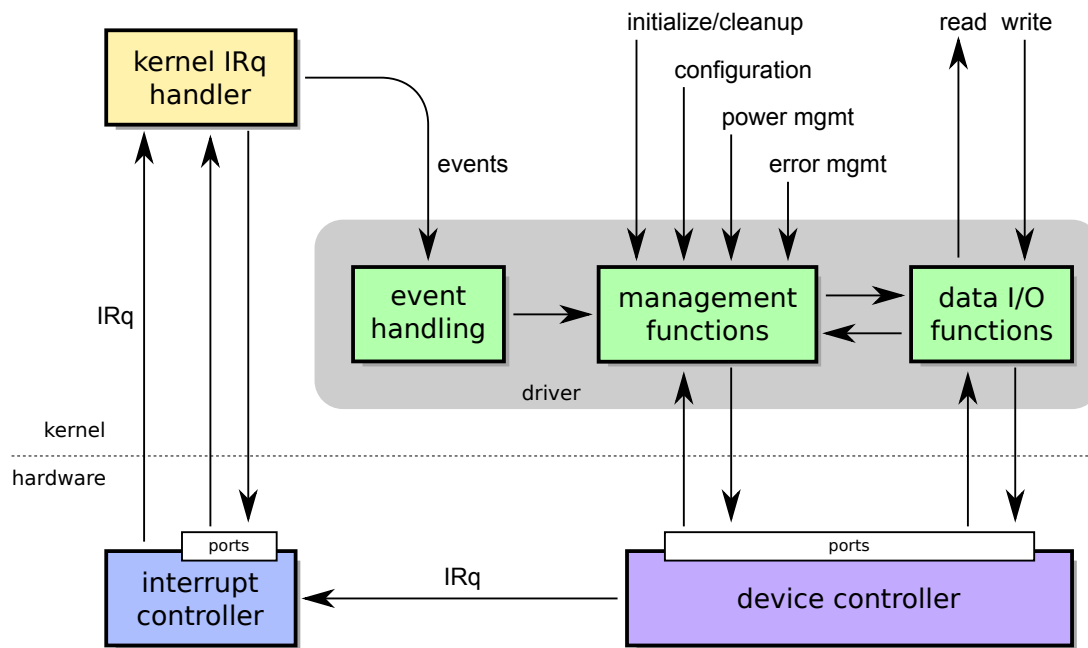


Figura 20.2: Visão geral de um *driver* de dispositivo.

20.5 Estratégias de interação

Cada *driver* deve interagir com seu respectivos dispositivo de entrada/saída para realizar as operações desejadas, através das portas do controlador do dispositivo. Esta seção aborda as três estratégias mais frequentemente usadas pelos *drivers* para essa interação, que são a entrada/saída controlada por programa, a controlada por eventos e o acesso direto à memória, detalhados a seguir.

20.5.1 Interação controlada por programa

A estratégia de entrada/saída mais simples, usada com alguns tipos de dispositivos, é a interação controlada por programa, também chamada **varredura**, **polling** ou **PIO** – *Programmed I/O*. Nesta abordagem, o *driver* solicita uma operação ao controlador do dispositivo, usando as portas *control* e *data-out* (ou *data-in*) de sua interface, e aguarda a conclusão da operação solicitada, monitorando continuamente os bits da respectiva porta de status.

Considerando as portas da interface paralela descrita na Seção 19.4, o comportamento do *driver* da interface paralela em uma operação de saída usando essa abordagem é descrito (simplificadamente) pelos trechos de código a seguir. O primeiro trecho de código contém definições de macros C usadas no restante do código:

```

1 // interface paralela LPT1
2 #define LPT1 0x0378 // endereço da interface paralela LPT1
3 #define DATA (LPT1 + 0) // porta de dados
4 #define STAT (LPT1 + 1) // porta de status
5 #define CTRL (LPT1 + 2) // porta de controle
6
7 // bits de controle e status
8 #define ACK 6 // bit 6 (porta de status)
9 #define BUSY 7 // bit 7 (porta de status)
10 #define STROBE 0 // bit 0 (porta de controle)
11
12 // operações em bits: seta/limpa/testa o N-ésimo bit de A
13 #define BIT_SET(A,N) (A | (1 << N)) // seta (= 1)
14 #define BIT_CLEAR(A,N) (A & ~(1 << N)) // limpa (= 0)
15 #define BIT_TEST(A,N) (A & (1 << N)) // testa

```

As leituras e escritas de bytes nas portas de E/S do controlador são representadas respectivamente pelas funções `inb` e `outb`, que encapsulam as instruções IN e OUT do processador:

```

1 // lê um byte da porta de E/S "port"
2 unsigned char inb (unsigned short int port) ;
3
4 // escreve o byte "value" na porta de E/S "port"
5 void outb (unsigned char value, unsigned short int port) ;

```

O código a seguir contém a implementação simplificada de uma operação de saída de caractere orientada a programa (*polling*); observa-se que foram omitidas a inicialização e o tratamento de erros:

```

1 void send_char_polling (unsigned char c)
2 {
3     // escreve o byte "c" a enviar na porta de dados
4     outb (c, DATA) ;
5
6     // espera a interface ficar livre (bit BUSY = 1)
7     while (! BIT_TEST (inb (STAT), BUSY)) ;
8
9     // gera um pulso 0 no bit STROBE (bit STROBE = 0) durante 1 us,
10    // para indicar à interface que há um novo dado em DATA
11    aux = inb (CTRL) ;
12    outb (BIT_CLEAR (aux, STROBE), CTRL) ; // bit STROBE = 0
13    usleep (1) ; // aguarda 1 us
14    outb (BIT_SET (aux, STROBE), CTRL) ; // bit STROBE = 1
15
16    // espera a interface receber o dado 3 (pulso em 0 no bit ACK)
17    while (BIT_TEST (inb (STAT), ACK)) ;
18 }

```

Pode-se perceber que o processador e o controlador executam ações coordenadas e complementares: o processador espera que o controlador esteja livre antes de lhe

³Este laço de espera sobre o bit ACK é desnecessário, pois o próximo envio de caractere testará o bit BUSY, que indica a conclusão da operação anterior. Contudo, ele foi mantido no código para maior clareza didática.

enviar um dado; por sua vez, o controlador espera que o processador lhe envie um novo dado para processar, e assim por diante. Essa interação é ilustrada na Figura 20.3.

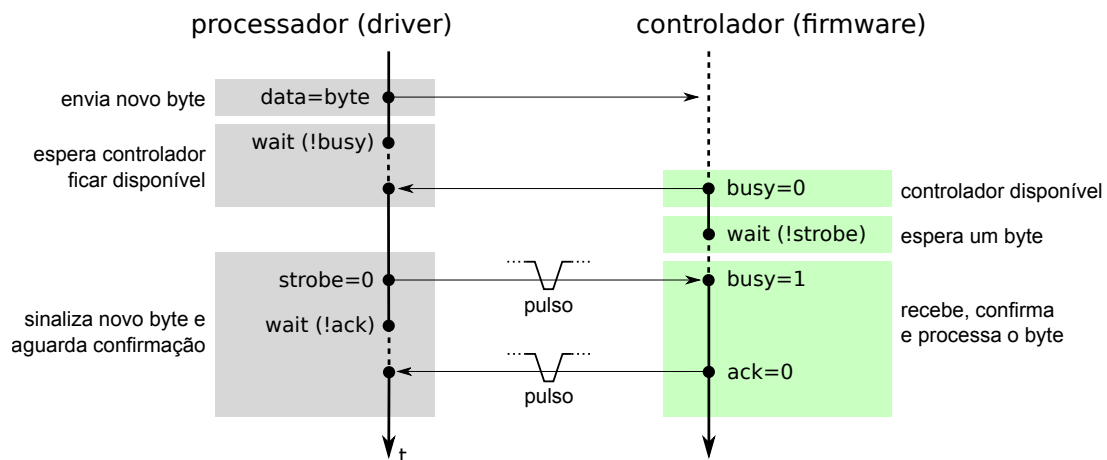


Figura 20.3: Entrada/saída controlada por programa.

Nessa estratégia, o controlador fica sempre esperando por novos dados, pois só precisa trabalhar quando há dados a processar, ou seja, quando o bit *strobe* de sua porta de controle indicar que há um novo dado em sua porta de dados. Por outro lado, manter o processador esperando até que a operação de E/S seja concluída é indesejável, sobretudo se essa operação for demorada. Além de constituir uma situação clássica de desperdício de recursos por **espera ocupada**, manter o processador esperando pela resposta de um controlador lento pode prejudicar o andamento de outras atividades importantes do sistema, como a interação com o usuário.

O problema da espera ocupada torna a estratégia de entrada/saída por programa pouco eficiente, sobretudo se o dispositivo for lento. Por isso, ela é pouco usada em sistemas operacionais de propósito geral. Seu uso se concentra sobretudo em sistemas embarcados dedicados, nos quais o processador só tem uma tarefa (ou poucas) a realizar. A estratégia básica de varredura pode ser modificada, substituindo o teste contínuo do status do dispositivo por um teste periódico (por exemplo, a cada 1ms), e permitindo ao processador executar outras tarefas enquanto o dispositivo estiver ocupado. Todavia, essa abordagem implica em uma menor taxa de transferência de dados para o dispositivo e, por essa razão, só é usada em dispositivos com baixa vazão de dados.

20.5.2 Interação controlada por eventos

Uma forma mais eficiente de interagir com dispositivos de entrada/saída consiste em efetuar a requisição da operação desejada e suspender a tarefa corrente, liberando o processador para tratar outras tarefas. Quando o dispositivo tiver terminado de processar a operação solicitada, seu controlador irá gerar uma requisição de interrupção (IRQ) para notificar o respectivo *driver*, que poderá então retomar a execução daquela tarefa. Essa estratégia de ação é denominada **interação controlada por eventos** ou por interrupções, pois as interrupções têm um papel fundamental em sua implementação.

Na estratégia de entrada/saída por eventos, uma operação de entrada ou saída é dividida em dois blocos de instruções: um bloco que inicia a operação, ativado pelo *driver* a pedido de um processo ou *thread*, e uma **rotina de tratamento de interrupção**

(*interrupt handler*), ativada a cada interrupção, para informar sobre a conclusão da última operação solicitada.

Considerando novamente a interface paralela descrita na Seção 19.4, o código a seguir representa o lançamento da operação de E/S pelo *driver* e a rotina de tratamento de interrupção (subentende-se as constantes e variáveis definidas na listagem anterior). Conforme visto na Seção 19.4, o controlador da interface paralela pode ser configurado para gerar uma interrupção através do flag *Enable_IRQ* de sua porta de controle (P_2).

```
1 // saída de dados por eventos (simplificado, sem controle de erros)
2 void send_char_irq (unsigned char c)
3 {
4     // escreve o byte "c" a enviar na porta de dados
5     outb (c, DATA) ;
6
7     // espera a interface ficar livre (bit BUSY = 1)
8     while (! BIT_TEST (inb (STAT), BUSY)) ;
9
10    // gera um pulso 0 no bit STROBE (bit STROBE = 0) durante 1 us,
11    // para indicar à interface que há um novo dado em DATA
12    aux = inb (CTRL) ;
13    outb (BIT_CLEAR (aux, STROBE), CTRL) ; // bit STROBE = 0
14    usleep (1) ;                          // aguarda 1 us
15    outb (BIT_SET (aux, STROBE), CTRL) ;   // bit STROBE = 1
16
17    // não espera o ACK da interface
18
19    // suspende a execução da tarefa atual e libera a CPU para outras tarefas
20    // enquanto a interface está ocupada processando o último dado recebido.
21    task_suspend (...) ;
22 }
23
24 // tratamento da interrupção gerada pela interface paralela
25 void irq_handle ()
26 {
27     // a interface concluiu sua operação e sinalizou com uma interrupção;
28     // acordar a tarefa que solicitou a última operação
29     task_awake (...) ;
30 }
```

Nesse exemplo, observa-se que o *driver* inicia a transferência de dados para a interface paralela e suspende a tarefa solicitante (chamada *task_suspend*), liberando o processador para outras tarefas. Ao ocorrer uma interrupção, a rotina de tratamento do *driver* (*irq_handle*) é ativada pelo SO e acorda a tarefa anteriormente suspensa. O diagrama da Figura 20.4 ilustra de forma simplificada a estratégia de entrada/saída usando interrupções.

Uma variante mais eficiente da operação por eventos consiste em fornecer ao *driver* um *buffer* com uma sequência de bytes a enviar, como mostra o código a seguir:

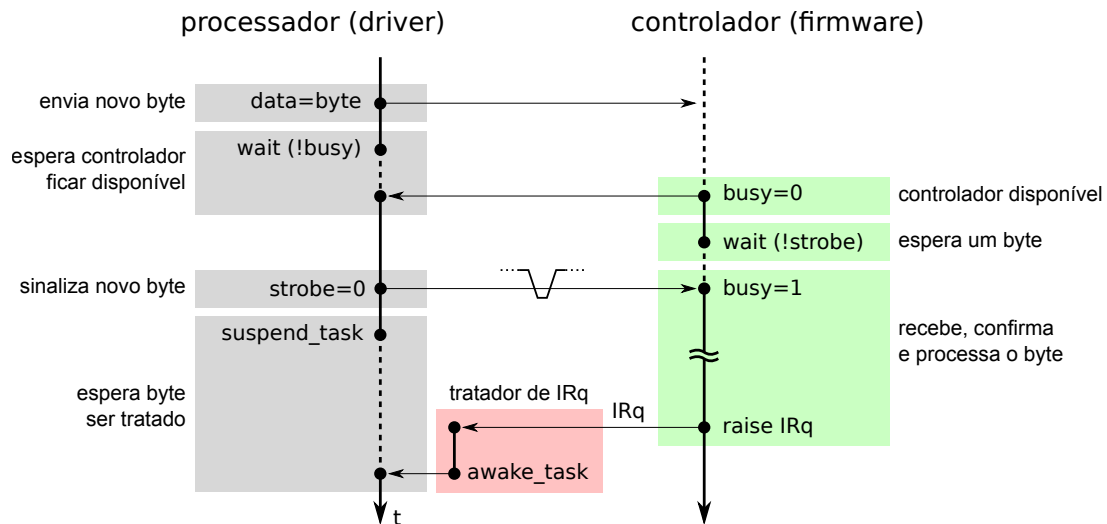


Figura 20.4: Entrada/saída controlada por eventos (interrupções).

```

1 // buffer de bytes a enviar
2 char *buffer ;
3 int buff_size, pos ;
4
5 // envia um buffer de dados de tamanho buff_size (simplificado)
6 void send_buffer (char *buffer)
7 {
8     // envia o primeiro byte do buffer
9     pos = 0 ;
10    send_char_irq (buffer[pos]) ;
11 }
12
13 // tratamento da interrupção gerada pela interface paralela
14 void irq_handle ()
15 {
16     pos++ ; // avança posição no buffer
17     if (pos >= buff_size) // o buffer terminou?
18         task_awake (...) ; // sim, acorda a tarefa
19     else
20         send_char_irq (buffer[pos]) ; // não, envia o próximo byte
21 }

```

20.5.3 Acesso direto à memória

Na maioria das vezes, o tratamento de operações de entrada/saída é uma operação lenta, pois os dispositivos periféricos são mais lentos que o processador. Além disso, o uso do processador principal para intermediar essas operações é ineficiente, pois implica em transferências adicionais (e desnecessárias) de dados: por exemplo, para transportar um byte de um *buffer* da memória para a interface paralela, esse byte precisa antes ser carregado em um registrador do processador, para em seguida ser enviado ao controlador da interface. Para resolver esse problema, a maioria dos computadores atuais, com exceção de pequenos sistemas embarcados dedicados, oferece mecanismos de **acesso direto à memória** (DMA - *Direct Memory Access*), que permitem transferências diretas entre a memória principal e os controladores de entrada/saída.

O funcionamento do mecanismo de acesso direto à memória em si é relativamente simples. Como exemplo, a seguinte sequência de passos seria executada para a escrita de dados de um *buffer* em memória RAM para o controlador de um disco rígido:

1. o processador acessa as portas do controlador de DMA associado ao dispositivo desejado, para informar o endereço inicial e o tamanho da área de memória RAM contendo os dados a serem escritos no disco. O tamanho da área de memória deve ser um múltiplo do tamanho dos blocos físicos do disco rígido (512 ou 4096 bytes);
2. o controlador de DMA solicita ao controlador do disco a transferência de dados da RAM para o disco e aguarda a conclusão da operação;
3. o controlador do disco recebe os dados da memória; essa operação pode ser repetida várias vezes, caso o volume de dados seja maior que o tamanho máximo de cada transferência;
4. ao final da transferência de dados, o controlador de DMA notifica o processador sobre a conclusão da operação, usando uma requisição de interrupção (IRQ).

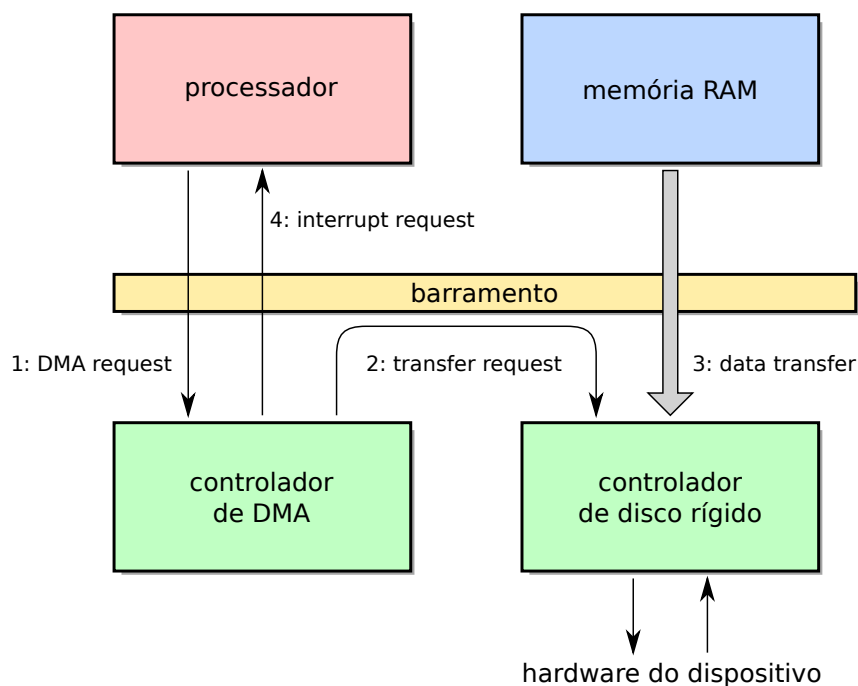


Figura 20.5: Funcionamento do acesso direto à memória.

A dinâmica dessas operações é ilustrada de forma simplificada na Figura 20.5. Uma vez efetuado o passo 1, o processador fica livre para outras tarefas⁴, enquanto o controlador de DMA e o controlador do disco se encarregam da transferência de dados

⁴Obviamente pode existir uma contenção (disputa) entre o processador e os controladores no acesso aos barramentos e à memória principal, mas esse problema é atenuado pelo fato do processador poder acessar o conteúdo das memórias *cache* enquanto a transferência DMA é executada. Além disso, circuitos de *arbitragem* intermedeiam o acesso à memória para evitar conflitos. Mais detalhes sobre contenção de acesso à memória durante operações de DMA podem ser obtidos em [Patterson and Hennessy, 2005].

propriamente dita. O código a seguir representa uma implementação hipotética de rotinas para executar uma operação de entrada/saída através de DMA.

```
1 // requisição da operação de saída através de DMA (simplificado)
2 void dma_output (...)
3 {
4     // solicita uma operação DMA, informando os dados da transferência
5     // através da estrutura "dma_data"
6     dma_request (dma_data) ;
7
8     // suspende a tarefa solicitante, liberando o processador
9     task_suspend (...) ;
10 }
11
12 // tratamento da interrupção gerada controlador de DMA
13 void irq_handle ()
14 {
15     // informa o controlador de interrupções que a IRq foi tratada
16     irq_acknowledge () ;
17
18     // saída terminou, acordar a tarefa solicitante
19     task_awake (...) ;
20 }
```

A implementação dos mecanismos de DMA depende da arquitetura e do barramento considerado. Computadores mais antigos dispunham de um controlador de DMA único, que oferecia vários **canais de DMA** distintos, permitindo a realização de transferências DMA simultâneas. Já os computadores pessoais usando barramento PCI não possuem um controlador DMA central; ao invés disso, cada controlador de dispositivo conectado ao barramento pode assumir o controle do mesmo para efetuar transferências DMA sem depender do processador principal [Corbet et al., 2005], gerenciando assim seu próprio canal.

No exemplo anterior, a ativação do mecanismo de DMA é dita **síncrona**, pois é feita explicitamente pelo processador, provavelmente em decorrência de uma chamada de sistema. Contudo, a ativação também pode ser **assíncrona**, quando ativada por um dispositivo de entrada/saída que dispõe de dados a serem transferidos para a memória, como ocorre com uma interface de rede ao receber dados provindos da rede.

O mecanismo de DMA é utilizado para transferir grandes blocos de dados diretamente entre a memória RAM e as portas dos dispositivos de entrada/saída, liberando o processador para outras tarefas. Todavia, como a configuração de cada operação de DMA é complexa, para pequenas transferências de dados acaba sendo mais rápido e simples usar o processador principal [Bovet and Cesati, 2005]. Por essa razão, o mecanismo de DMA é usado sobretudo nas operações de entrada/saída envolvendo dispositivos que produzem ou consomem grandes volumes de dados, como discos, interfaces de rede, entradas e saídas de áudio e interfaces gráficas.

20.6 Tratamento de interrupções

Durante a execução de uma rotina de tratamento de interrupção, é usual inibir novas interrupções, para evitar a execução aninhada de tratadores de interrupção, o que tornaria o código dos *drivers* (e do núcleo) bem mais complexo e suscetível a erros.

Entretanto, manter interrupções inibidas durante muito tempo pode ocasionar perdas de dados ou outros problemas. Por exemplo, uma interface de rede gera uma interrupção quando recebe um pacote vindo da rede; esse pacote fica em seu buffer interno e deve ser transferido dali para a memória principal antes que outros pacotes cheguem, pois esse buffer tem uma capacidade limitada. Por essa razão, o tratamento das interrupções deve ser feito de forma muito rápida.

Para obter rapidez, a maioria dos sistemas operacionais implementa o tratamento de interrupções em dois níveis: um **tratador primário** (FLIH - *First-Level Interrupt Handler*) e um **tratador secundário** (SLIH - *Second-Level Interrupt Handler*). O tratador primário, também chamado *hard/fast interrupt handler* ou ainda *top-half handler*, é ativado a cada interrupção recebida e executa rapidamente, com as demais interrupções desabilitadas. Sua tarefa consiste em reconhecer a ocorrência da interrupção junto ao controlador de interrupções, criar um *descriptor de evento* contendo os dados da interrupção ocorrida, inserir esse descriptor em uma fila de eventos pendentes junto ao *driver* do respectivo dispositivo e notificar o *driver*.

O tratador secundário, também conhecido como *soft/slow interrupt handler* ou ainda *bottom-half handler*, tem por objetivo tratar os eventos pendentes registrados pelo tratador primário. Ele geralmente é escalonado como uma *thread* de núcleo com alta prioridade, executando quando um processador estiver disponível. Embora mais complexa, esta estrutura em dois nível traz vantagens: ao permitir um tratamento mais rápido de cada interrupção, ela minimiza o risco de perder interrupções simultâneas; além disso, a fila de eventos pendentes pode ser analisada para remover eventos redundantes (como atualizações consecutivas de posição do *mouse*). A Figura 20.6 ilustra essa estrutura e a sequência de ações envolvidas no tratamento de cada interrupção.

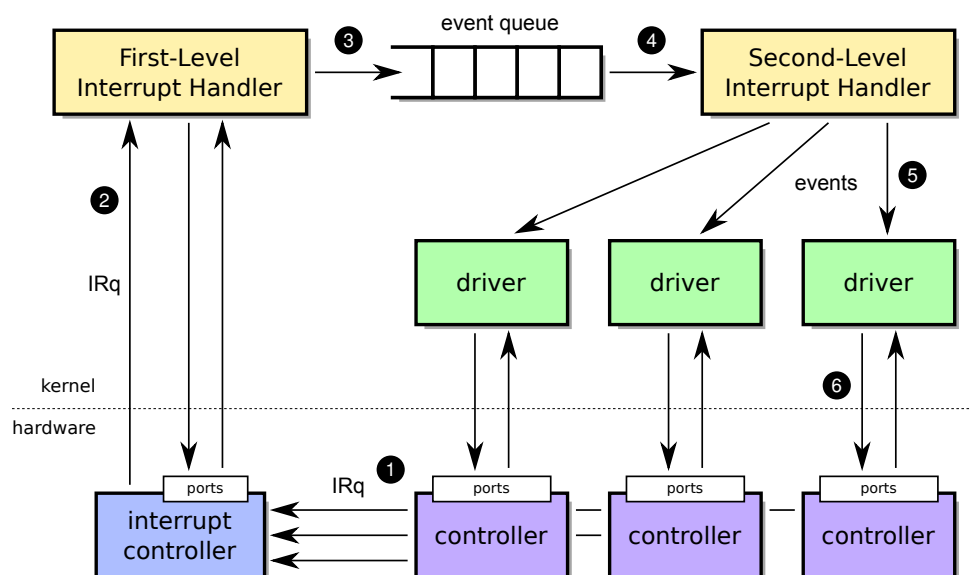


Figura 20.6: Estrutura dos tratadores de interrupções.

No Linux, cada interrupção possui sua própria fila de eventos pendentes e seus próprios *top-half* e *bottom-half*. Os tratadores secundários são lançados pelos respectivos tratadores primários, sob a forma de *threads* de núcleo especiais (denominadas *tasklets* ou *workqueues*) [Bovet and Cesati, 2005]. O núcleo Windows NT e seus sucessores implementam o tratamento primário através de rotinas de serviço de interrupção (ISR - *Interrupt Service Routine*). Ao final de sua execução, cada ISR agenda o tratamento

secundário da interrupção através de um procedimento postergado (DPC - *Deferred Procedure Call*) [Russinovich et al., 2008]. O sistema *Symbian* usa uma abordagem similar a esta.

Por outro lado, os sistemas Solaris, FreeBSD e MacOS X usam uma abordagem denominada *interrupt threads* [Mauro and McDougall, 2006]. Cada interrupção provoca o lançamento de uma *thread* de núcleo, que é escalonada e compete pelo uso do processador de acordo com sua prioridade. As interrupções têm prioridades que podem estar acima da prioridade do escalonador ou abaixo dela. Como o próprio escalonador também é uma *thread*, interrupções de baixa prioridade podem ser interrompidas pelo escalonador ou por outras interrupções. Por outro lado, interrupções de alta prioridade não são interrompidas pelo escalonador, por isso devem executar rapidamente.

Exercícios

1. Explique o que é e como funciona o tratamento de interrupções em dois níveis pelo sistema operacional.
2. Considere um dispositivo físico cujo controlador tem quatro portas de 8 bits:

porta	sentido	função	valor
<i>data_in</i>	<i>dev</i> → <i>cpu</i>	leitura de dados	byte lido
<i>data_out</i>	<i>cpu</i> → <i>dev</i>	escrita de dados	byte a escrever
<i>status</i>	<i>dev</i> → <i>cpu</i>	estado do dispositivo	IDLE, BUSY ou ERROR
<i>control</i>	<i>cpu</i> → <i>dev</i>	operação a efetuar	READ, WRITE ou RESET

Essas portas são acessadas através das operações “*val=in(port)*” e “*out(port, val)*”.

Escreva em pseudocódigo as funções “*char readchar()*” e “*void writechar(char c)*” do *driver* desse dispositivo, que implementam respectivamente as operações de **leitura** e de **escrita** de caracteres individuais, usando a estratégia de *polling* (interação por programa).

Referências

- D. Bovet and M. Cesati. *Understanding the Linux Kernel*, 3rd edition. O'Reilly Media, Inc, 2005.
- J. Corbet, A. Rubini, and G. Kroah-Hartman. *Linux Device Drivers*, 3rd Edition. O'Reilly Media, Inc, 2005.
- J. Mauro and R. McDougall. *Solaris Internals: Solaris 10 and OpenSolaris Kernel Architecture*. Prentice-Hall PTR, 2006.
- D. Patterson and J. Henessy. *Organização e Projeto de Computadores*. Campus, 2005.
- M. Russinovich, D. Solomon, and A. Ionescu. *Microsoft Windows Internals*, Fifth Edition. Microsoft Press, 2008.